MCP For Unity 문서

목차

- 一、소개
- 二、사용 방법
  - 2.1 에디터 사용
  - 2.2 런타임 사용
- 三、확장성 설명
  - 3.1 에디터 확장
  - 3.2 런타임 확장
- 四、부록
- 五、AI 보조 개발

一、소개

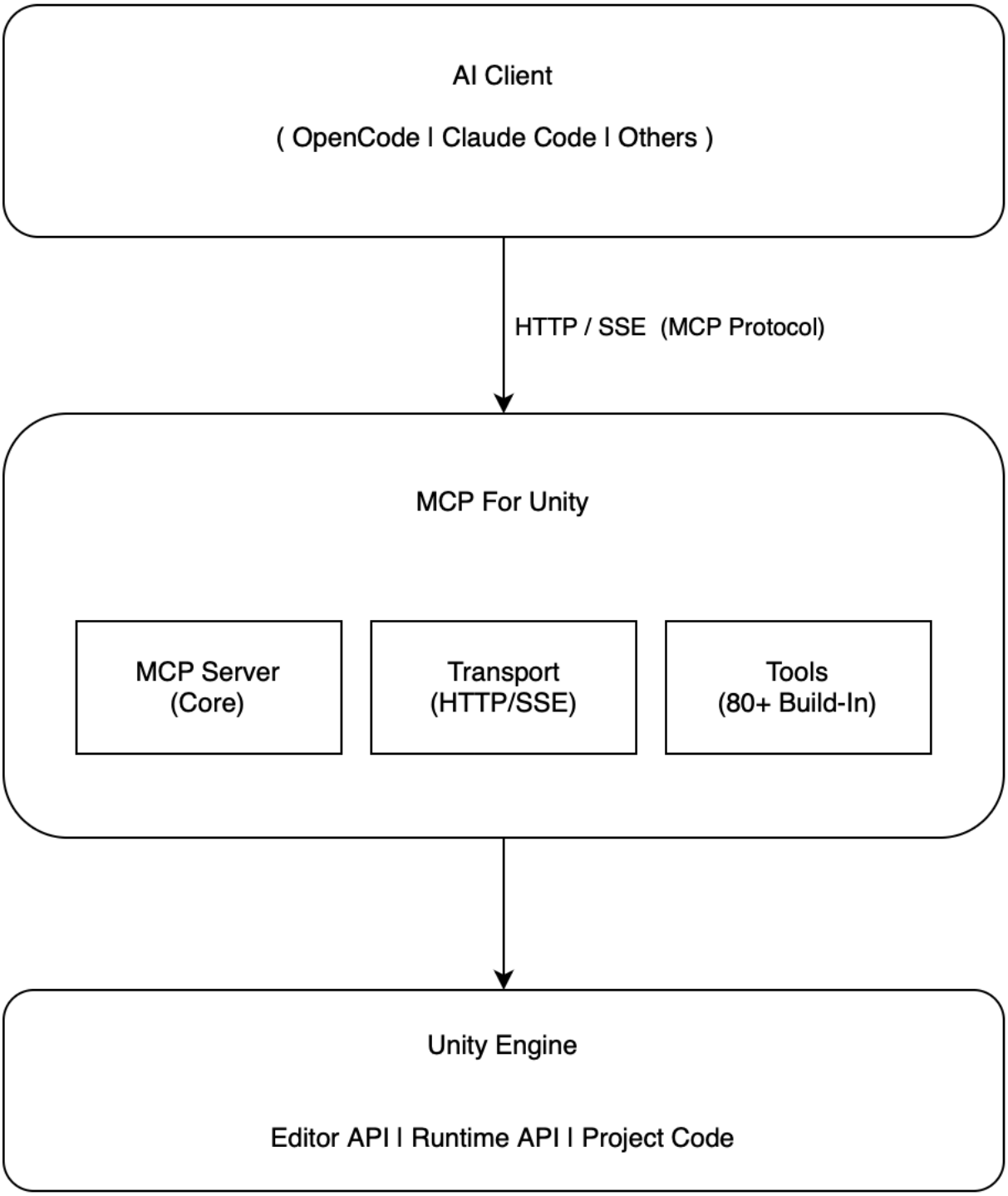
1.1 MCP For Unity 란

MCP For Unity는 **Model Context Protocol (MCP)** 기반의 Unity 구현 프레임워크로, Unity 에디터와 런타임 애플리케이션을 AI 생태계에 원활하게 통합하는 것을 목적으로 합니다.

핵심 장점:

특징	설명
순수 C# 구현	외부 브리지 불필요, Python이나 다른 언어 의존성 없음
전체 툴체인 접근	AI가 Unity Engine API와 프로젝트 코드를 직접 호출 가능
양방향 통신	AI와 Unity 간의 실시간 상호작용 지원
표준화된 프로토콜	Claude Desktop, VS Code MCP 등 주요 AI 클라이언트와 호환

1.2 아키텍처 개요


AI Client
(OpenCode I Claude Code I Others)
HTTP / SSE (MCP Protocol)
MCP For Unity
MCP Server
(Core)
Transport
(HTTP/SSE)
Tools
(80+ Build-In)
Unity Engine
Editor API I Runtime API I Project Code

1.3 기능 지원

기능	지원 상태
Tools (도구)	✅ 완전 지원
Resources (리소스)	✅ 완전 지원
Tasks (작업)	✅ 완전 지원
Prompts (프롬프트)	❌ 미지원

> ⚠️ **참고**: 현재 버전에서는 Prompts 기능이 지원되지 않습니다.

1.4 기술 사양

항목	설명
Unity 버전	2021.3+
.NET 버전	.NET Standard 2.1
전송 프로토콜	HTTP + Server-Sent Events (SSE)
직렬화	Newtonsoft.Json
의존성	`com.unity.nuget.newtonsoft-json`

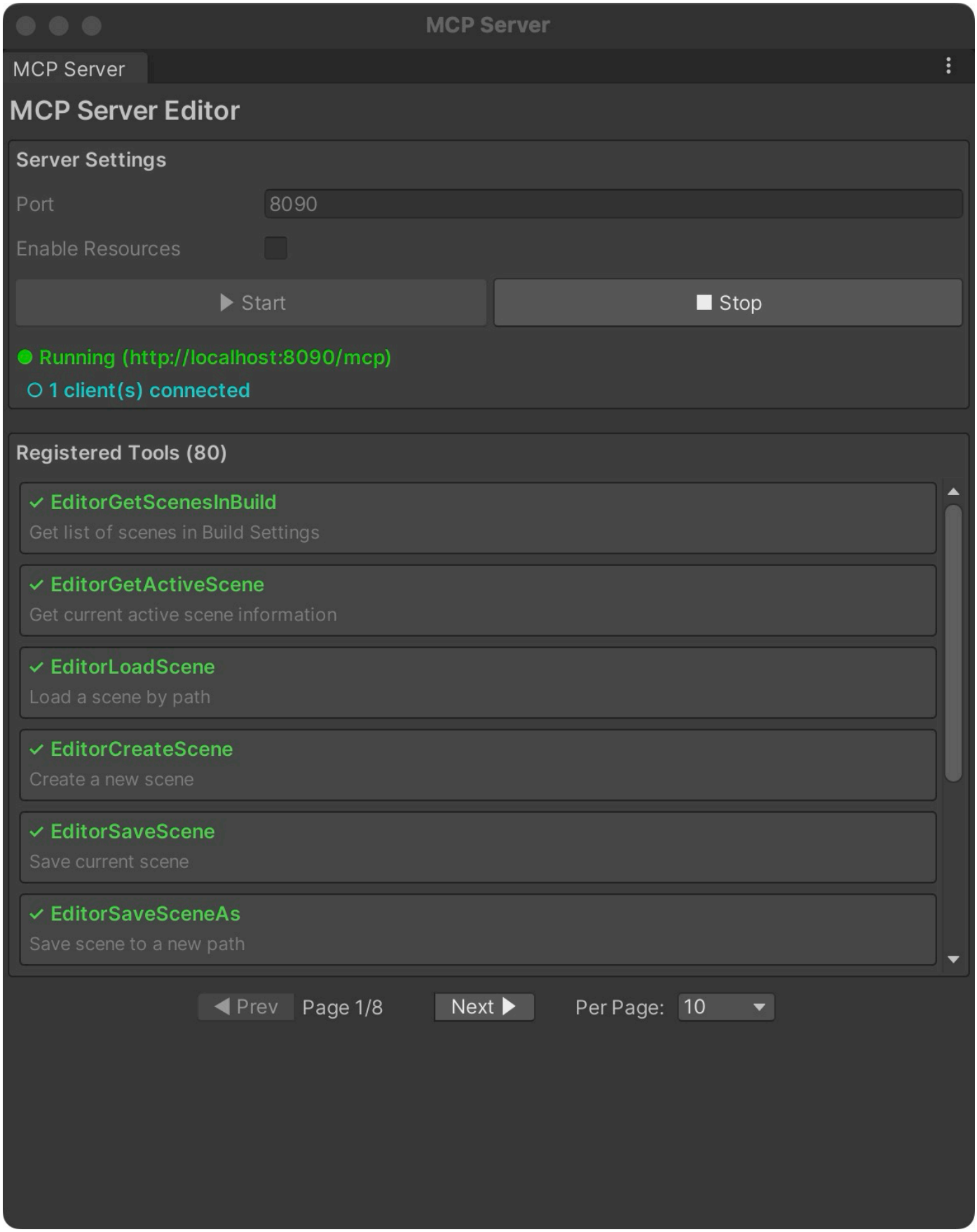
二、사용 방법

2.1 에디터 사용

2.1.1 에디터 서버 시작

방법 1: 에디터 윈도우에서

1. 메뉴 열기: `Tools > MCP For Unity > Server Window`
2. 서버 매개변수 설정
3. **Start** 버튼 클릭하여 시작


MCP Server
MCP Server
MCP Server Editor
Server Settings
Port
8090
Enable Resources
Start
Stop
Running (http://localhost:8090/mcp)
1 client(s) connected
Registered Tools (80)
EditorGetScenesInBuild
Get list of scenes in Build Settings
EditorGetActiveScene
Get current active scene information
EditorLoadScene
Load a scene by path
EditorCreateScene
Create a new scene
EditorSaveScene
Save current scene
EditorSaveSceneAs
Save scene to a new path
Prev
Page 1/8
Next
Per Page:
10

방법 2: 코드에서

```
using ModelContextProtocol.Editor;

// 시작 (기본 설정 사용)
GlobalEditorMcpServer.StartServer();

// 중지
GlobalEditorMcpServer.StopServer();
```

2.1.2 설정 옵션

에디터 윈도우에서 다음 옵션을 설정할 수 있습니다:

옵션	기본값	설명
Port	8090	서버 포트
Enable Resources	false	리소스 서비스 활성화
Enable File Watching	false	파일 감시 활성화

2.1.3 내장 도구 개요

MCP For Unity는 **80개의 에디터 도구**를 제공하며, 일반적인 개발 시나리오를 다룹니다:

씬 관리 (10개)

도구 이름	설명
`EditorGetScenesInBuild`	Build Settings의 씬 목록 가져오기
`EditorGetActiveScene`	현재 활성 씬 정보 가져오기
`EditorLoadScene`	씬 로드
`EditorCreateScene`	새 씬 생성
`EditorSaveScene`	현재 씬 저장
`EditorSaveSceneAs`	씬 다른 이름으로 저장
`EditorCloseScene`	씬 닫기
`EditorAddSceneToBuild`	씬을 Build에 추가
`EditorRemoveSceneFromBuild`	씬을 Build에서 제거
`EditorGetSceneHierarchy`	씬 계층 구조 가져오기

GameObject 작업 (15개)

도구 이름	설명
`EditorCreateGameObject`	빈 GameObject 생성
`EditorCreatePrimitive`	기본 도형 생성
`EditorDuplicateGameObject`	GameObject 복제
`EditorDeleteGameObject`	GameObject 삭제
`EditorFindGameObject`	GameObject 찾기
`EditorFindGameObjectsByName`	이름으로 찾기
`EditorFindGameObjectsByTag`	태그로 찾기
`EditorFindGameObjectsByLayer`	레이어로 찾기
`EditorGetGameObjectInfo`	상세 정보 가져오기
`EditorGetChildren`	자식 목록 가져오기
`EditorSetParent`	부모 설정
`EditorSetTag`	태그 설정
`EditorSetLayer`	레이어 설정
`EditorSetStatic`	스태틱 플래그 설정
`EditorSetActive`	활성 상태 설정

Transform 작업 (10개)

도구 이름	설명
`EditorSetPosition`	월드 위치 설정
`EditorSetRotation`	월드 회전 설정
`EditorSetScale`	스케일 설정
`EditorSetLocalPosition`	로컬 위치 설정
`EditorSetLocalRotation`	로컬 회전 설정
`EditorSetLocalScale`	로컬 스케일 설정
`EditorSetPositionAndRotation`	위치와 회전 동시 설정
`EditorTranslate`	상대 이동
`EditorRotate`	상대 회전
`EditorResetTransform`	트랜스폼 리셋

컴포넌트 작업 (10개)

도구 이름	설명
`EditorAddComponent`	컴포넌트 추가
`EditorRemoveComponent`	컴포넌트 제거
`EditorGetComponent`	컴포넌트 정보 가져오기
`EditorGetComponents`	모든 컴포넌트 가져오기
`EditorGetComponentsInChildren`	자식 오브젝트의 컴포넌트 가져오기
`EditorSetComponentProperty`	컴포넌트 속성 설정
`EditorGetComponentProperty`	컴포넌트 속성 가져오기
`EditorHasComponent`	컴포넌트 존재 여부 확인
`EditorGetRequiredComponent`	컴포넌트 가져오기 또는 추가
`EditorSendComponentMessage`	메시지 전송

컴파일 & 빌드 (10개)

도구 이름	설명
`EditorGetCompilationStatus`	컴파일 상태 가져오기
`EditorIsCompiling`	컴파일 중인지 확인
`EditorGetCompileErrors`	컴파일 에러 가져오기
`EditorGetCompileWarnings`	컴파일 경고 가져오기
`EditorGetBuildTarget`	빌드 타겟 가져오기
`EditorSetBuildTarget`	빌드 타겟 설정
`EditorGetScriptingBackend`	스크립팅 백엔드 가져오기
`EditorGetAssemblyDefinitions`	asmdef 목록 가져오기
`EditorGetDefineSymbols`	정의 심볼 가져오기
`EditorSetDefineSymbols`	정의 심볼 설정

에셋 관리 (15개)

도구 이름	설명
`EditorFindAssets`	에셋 검색
`EditorGetAssetInfo`	에셋 상세 정보 가져오기
`EditorGetAssetPath`	에셋 경로 가져오기
`EditorGetAssetDependencies`	에셋 의존성 가져오기
`EditorGetFolderContents`	폴더 내용 가져오기
`EditorCreateFolder`	폴더 생성
`EditorDeleteAsset`	에셋 삭제
`EditorRenameAsset`	에셋 이름 변경
`EditorMoveAsset`	에셋 이동
`EditorCopyAsset`	에셋 복사
`EditorRefreshAssets`	에셋 데이터베이스 새로고침
`EditorReimportAsset`	에셋 재임포트
`EditorGetAssetImportSettings`	임포트 설정 가져오기
`EditorLoadAssetAtPath`	경로에서 에셋 로드
`EditorGetAllAssetPaths`	모든 에셋 경로 가져오기

프리팹 작업 (10개)

도구 이름	설명
`EditorInstantiatePrefab`	프리팹 인스턴스화
`EditorCreatePrefab`	프리팹 생성
`EditorApplyPrefab`	프리팹 변경 사항 적용
`EditorRevertPrefab`	프리팹 되돌리기
`EditorUnpackPrefab`	프리팹 언팩
`EditorGetPrefabInfo`	프리팹 정보 가져오기
`EditorGetPrefabType`	프리팹 타입 가져오기
`EditorIsPrefabInstance`	프리팹 인스턴스인지 확인
`EditorGetPrefabAssetPath`	프리팹 에셋 경로 가져오기
`EditorGetAllPrefabs`	모든 프리팹 가져오기

2.1.4 구현 위치

```
Assets/McpForUnity/Editor/
├── Tools/
│   └── EditorToolsList.cs       # 80개 에디터 도구 구현
├── GlobalEditorMcpServer.cs     # 글로벌 서버 관리
└── Window/
    └── McpServerEditorWindow.cs # 에디터 윈도우
```

2.1.5 AI 클라이언트 연결

Claude Desktop 설정

Claude Desktop 설정 파일 편집:

- macOS: `~/Library/Application Support/Claude/claude_desktop_config.json`
- Windows: `%APPDATA%\Claude\claude_desktop_config.json`

```json
{
  "mcpServers": {
    "unity-editor": {
      "url": "http://localhost:8090/mcp",
      "transport": "sse"
    }
  }
}
```

OpenCode 설정

프로젝트 루트에 `.opencode/opencode.jsonc` 파일 생성:

```json
{
  "$schema": "https://opencode.ai/config.json",
  "mcp": {
    "UnityMCPServer": {
      "type": "remote",
      "url": "http://localhost:8090/mcp",
      "enabled": true
    }
  }
}
```

2.2 런타임 사용

2.2.1 런타임 서버 시작

```
using System.Threading.Tasks;
using ModelContextProtocol.Unity;
using UnityEngine;
```

```
public class MCPManager : MonoBehaviour
{
    private McpServerHost _host;

    async void Start()
    {
        // 설정 생성
        var options = new McpServerHostOptions
        {
            Port = 3000,
            ServerName = "UnityMCP",
            ServerVersion = "1.0.0"
        };

        // 서버 생성 및 시작
        _host = new McpServerHost(options);
        await _host.StartAsync();

        // 커스텀 도구 등록
        _host.Server.RegisterToolsFromClass(typeof(MyGameTools));

        Debug.Log($"MCP Server started at http://localhost:{options.Port}/mcp");
    }

    async void OnDestroy()
    {
        if (_host != null)
        {
            await _host.DisposeAsync();
        }
    }
}
```

> ⚠️ **중요**: 런타임 서버에는 **내장 도구가 포함되어 있지 않습니다**. 모든 도구는 `RegisterToolsFromClass()` 또는 `AddCustomTool()`을 사용하여 수동으로 등록해야 합니다.

2.2.2 설정 옵션

옵션	타입	기본값	설명
`Port`	int	3000	서버 포트
`ServerName`	string	"UnityMCP"	서버 이름
`ServerVersion`	string	"1.0.0"	서버 버전
`Instructions`	string	-	서버 설명
`LogLevel`	LogLevel	Information	로그 레벨

2.2.3 이벤트 구독

```
_host.OnServerStarted += () => Debug.Log("Server started");
_host.OnServerStopped += () => Debug.Log("Server stopped");
_host.OnServerError += (error) => Debug.LogError($"Server error: {error}");
```

2.2.4 커스텀 도구 등록

```
// 방법 1: 클래스를 통해 등록
_host.Server.RegisterToolsFromClass(typeof(MyGameTools));

// 방법 2: Lambda 직접 추가
_host.AddCustomTool("my_tool", "도구 설명", async (args, ct) =>
{
    string param = args?["param"]?.ToString();
    // 처리 로직
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = "결과" }
        }
    };
});
```

三、확장성 설명

3.1 에디터 확장

3.1.1 커스텀 도구 정의

속성 어노테이션을 사용하여 도구 정의:

```
using ModelContextProtocol.Protocol;
using ModelContextProtocol.Server;

public static class MyEditorTools
{
    [McpServerTool("EditorMyCustomTool", Description = "나의 커스텀 에디터 도구")]
    public static CallToolResult MyCustomTool(
        [McpArgument(Description = "매개변수 1", Required = true)] string param1,
        [McpArgument(Description = "매개변수 2 (선택)")] int param2 = 0)
    {
        // 에디터 작업 실행
        Debug.Log($"커스텀 도구 실행: {param1}, {param2}");

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"성공: {param1}" }
```

```
            }
        };
    }
}
```

도구 등록:

```
// GlobalEditorMcpServer 시작 후 등록
GlobalEditorMcpServer.Server.RegisterToolsFromClass(typeof(MyEditorTools));
```

3.1.2 매개변수 정의 방법

기본 타입 매개변수

```
[McpServerTool("Example", Description = "예제")]
public static CallToolResult Example(
    [McpArgument(Description = "문자열 매개변수", Required = true)] string text,
    [McpArgument(Description = "정수 매개변수")] int number = 0,
    [McpArgument(Description = "부동소수점 매개변수")] float value = 0f,
    [McpArgument(Description = "불리언 매개변수")] bool flag = false)
{
    // ...
}
```

열거형 매개변수

```
public enum MyEnum { OptionA, OptionB, OptionC }

[McpServerTool("EnumExample", Description = "열거형 예제")]
public static CallToolResult EnumExample(
    [McpArgument(Description = "열거형 옵션")] MyEnum option = MyEnum.OptionA)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"선택됨: {option}" }
        }
    };
}
```

배열 매개변수

```
[McpServerTool("ArrayExample", Description = "배열 예제")]
public static CallToolResult ArrayExample(
    [McpArgument(Description = "문자열 배열", Required = true)] string[] items)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"{items.Length}개 항목 수신" }
        }
    };
}
```

Vector3 매개변수 (확장 형식)

```
[McpServerTool("VectorExample", Description = "Vector3 예제")]
public static CallToolResult VectorExample(
    [McpArgument(Description = "위치")] UnityEngine.Vector3 position)
{
    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"위치: {position}" }
        }
    };
}
```

호출 시 매개변수 형식:

```
{
  "position_x": 1.0,
  "position_y": 2.0,
  "position_z": 3.0
}
```

3.1.3 커스텀 타입 정의

커스텀 타입 정의

```
using Newtonsoft.Json;
using ModelContextProtocol.Server;

public class PlayerData
{
    [JsonProperty("playerName")]
    [McpArgument(Description = "플레이어 이름", Required = true)]
    public string PlayerName;

    [JsonProperty("level")]
```

```
    [McpArgument(Description = "플레이어 레벨")]
    public int Level;

    [JsonProperty("health")]
    [McpArgument(Description = "체력")]
    public float Health;

    [JsonProperty("position")]
    [McpArgument(Description = "위치")]
    public UnityEngine.Vector3 Position;
}
```

도구에서 커스텀 타입 사용

```
[McpServerTool("SetPlayerData", Description = "플레이어 데이터 설정")]
public static CallToolResult SetPlayerData(
    [McpArgument(Description = "플레이어 데이터", Required = true)] PlayerData data)
{
    Debug.Log($"플레이어: {data.PlayerName}, 레벨: {data.Level}");

    return new CallToolResult
    {
        Content = new List<ContentBlock>
        {
            new TextContentBlock { Text = $"플레이어 설정됨: {data.PlayerName}" }
        }
    };
}
```

호출 형식:

```
{
  "data": {
    "playerName": "Player1",
    "level": 10,
    "health": 100.0,
    "position_x": 0,
    "position_y": 1,
    "position_z": 0
  }
}
```

커스텀 타입 검증 규칙

규칙	설명
필드는 `[JsonProperty]`와 `[McpArgument]` 모두 필요	어느 하나의 속성이 누락된 필드는 무시됨
`[JsonRequired]` 또는 `[McpArgument(Required = true)]` 사용	필수 필드 표시
중첩 커스텀 타입 지원	재귀적 검증
커스텀 타입 배열 지원	`PlayerData[]` 또는 `List<PlayerData>`

3.1.4 인스턴스 도구 등록

인스턴스 도구를 사용하면 동일한 클래스의 여러 인스턴스에 독립적인 도구 세트를 등록할 수 있습니다.

인스턴스 도구 클래스 정의

```
[McpInstanceTool(Name = "Player", Description = "플레이어 인스턴스 도구")]
public class PlayerTools
{
    public string PlayerId { get; set; }
    public int Health { get; set; }
    public int Score { get; set; }

    [McpServerTool(Description = "플레이어 정보 가져오기")]
    public CallToolResult GetInfo()
    {
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"플레이어 {PlayerId}: HP={Health}, Score=
{Score}" }
            }
        };
    }

    [McpServerTool(Description = "체력 설정")]
    public CallToolResult SetHealth(
        [McpArgument(Description = "체력 값", Required = true)] int health)
    {
        Health = health;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"HP를 {Health}(으)로 설정" }
            }
        };
    }
```

```
    [McpServerTool(Description = "점수 추가")]
    public void AddScore(
        [McpArgument(Description = "점수량", Required = true)] int amount)
    {
        Score += amount;
    }
}
```

인스턴스 도구 등록

```
// 인스턴스 생성
var player1 = new PlayerTools { PlayerId = "P001", Health = 100, Score = 0 };
var player2 = new PlayerTools { PlayerId = "P002", Health = 80, Score = 50 };

// 서버에 등록
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player1, "player_1");
GlobalEditorMcpServer.Server.RegisterToolsFromInstance(player2, "player_2");

// 도구 이름 형식: {instanceId}.{methodName}
// player_1.GetInfo
// player_1.SetHealth
// player_2.GetInfo
// player_2.SetHealth
```

인스턴스 도구 등록 해제

```
// 지정된 인스턴스의 모든 도구 등록 해제
GlobalEditorMcpServer.Server.UnregisterInstanceTools("player_1");
```

주의사항

- 인스턴스 ID는 고유해야 합니다
- 서버가 인스턴스 참조를 보유하므로 메모리 관리에 주의
- 더 이상 필요하지 않을 때 등록 해제

3.2 런타임 확장

런타임 확장은 에디터 확장과 동일한 방식으로 작동합니다.

3.2.1 커스텀 도구 정의

```
public static class GameTools
{
    [McpServerTool("SpawnEnemy", Description = "적 생성")]
    public static CallToolResult SpawnEnemy(
        [McpArgument(Description = "적 타입")] string enemyType,
        [McpArgument(Description = "위치")] UnityEngine.Vector3 position)
    {
```

```
        // 게임 런타임 로직
        var enemy = SpawnManager.SpawnEnemy(enemyType, position);

        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = $"적 생성됨: {enemy.name}" }
            }
        };
    }

    [McpServerTool("GetPlayerStatus", Description = "플레이어 상태 가져오기")]
    public static CallToolResult GetPlayerStatus()
    {
        var player = GameManager.Instance.Player;
        return new CallToolResult
        {
            Content = new List<ContentBlock>
            {
                new TextContentBlock { Text = JObject.FromObject(new
                {
                    health = player.Health,
                    mana = player.Mana,
                    level = player.Level
                }).ToString() }
            }
        };
    }
}
```

등록:

```
_host.Server.RegisterToolsFromClass(typeof(GameTools));
```

3.2.2 인스턴스 도구 등록

```
// 정의
[McpInstanceTool(Name = "Unit", Description = "유닛 도구")]
public class UnitTools
{
    public Unit Unit { get; set; }

    [McpServerTool(Description = "타겟으로 이동")]
    public CallToolResult MoveTo(
        [McpArgument(Description = "타겟 위치")] UnityEngine.Vector3 target)
    {
        Unit.MoveTo(target);
        return new CallToolResult
        {
            Content = new List<ContentBlock>
```

```
            {
                new TextContentBlock { Text = "이동 명령 전송됨" }
            }
        };
    }
}

// 등록
var unit = new UnitTools { Unit = myUnit };
_host.Server.RegisterToolsFromInstance(unit, "unit_001");

// 등록 해제
_host.Server.UnregisterInstanceTools("unit_001");
```

四、부록

4.1 McpServerHostOptions 설정 표

속성	타입	기본값	설명
`Port`	int	3000	HTTP 서비스 포트
`ServerName`	string	"UnityMCP"	MCP 서버 이름
`ServerVersion`	string	"1.0.0"	서버 버전
`Instructions`	string	-	AI에 전송할 지침
`LogLevel`	LogLevel	Information	로그 출력 레벨

4.2 이벤트 목록

이벤트	시그니처	발생 시점
`OnServerStarted`	`Action`	서버 시작 성공 시
`OnServerStopped`	`Action`	서버 중지 시
`OnServerError`	`Action<string>`	서버 에러 발생 시

4.3 반환 결과 타입

성공 결과

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "작업 성공" }
    }
};
```

에러 결과

```
return new CallToolResult
{
    IsError = true,
    Content = new List<ContentBlock>
    {
        new TextContentBlock { Text = "에러 메시지" }
    }
};
```

이미지 포함 결과

```
return new CallToolResult
{
    Content = new List<ContentBlock>
    {
        new ImageContentBlock
        {
            Data = base64ImageData,
            MimeType = "image/png"
        }
    }
};
```

4.4 주의사항

1. **스레드 안전**: MCP 도구는 백그라운드 스레드에서 실행됩니다. Unity 객체를 조작할 때는 `MainThreadDispatcher`를 사용하거나 메인 스레드 실행을 확인하세요
2. **라이프사이클**: `McpServerHost`는 자동으로 `Application.quitting`을 수신하고 애플리케이션 종료 시 정리합니다
3. **보안**: 기본적으로 `localhost`에서 수신하며 외부 연결을 허용하지 않습니다. 프로덕션에서는 신중하게 구성하세요
4. **성능**: 과도한 도구 호출은 성능에 영향을 줄 수 있으므로 호출 빈도를 적절히 관리하세요
5. **에러 처리**: 도구 내부 예외는 캡처되어 에러 결과로 반환되며, 서버는 충돌하지 않습니다

4.5 파일 구조

```
Assets/McpForUnity/
├── Core/
```

```
│   ├── IMcpServerHost.cs          # 서버 인터페이스
│   ├── McpServerHost.cs           # 서버 구현
│   ├── McpServerHostOptions.cs    # 설정 클래스
│   ├── McpException.cs            # 예외 타입
│   ├── Throw.cs                   # 매개변수 검증
│   └── Protocol/                  # MCP 프로토콜 타입
├── Server/
│   ├── McpServer.cs               # 코어 서버
│   ├── McpServerOptions.cs        # 서버 설정
│   └── Tools/
│       └── Attributes.cs          # 도구 속성 정의
├── Transport/
│   ├── ITransport.cs              # 전송 인터페이스
│   └── HttpListenerServerTransport.cs
├── Editor/
│   ├── Tools/
│   │   └── EditorToolsList.cs     # 80개 에디터 도구
│   ├── GlobalEditorMcpServer.cs   # 에디터 서버
│   └── Window/
│       └── McpServerEditorWindow.cs
├── Utilities/
│   ├── UnityLogger.cs             # 로그 구현
│   └── MainThreadDispatcher.cs    # 메인 스레드 디스패치
└── Samples/
    └── MCPExampleUsage.cs         # 사용 예제
```

4.6 자주 묻는 질문

Q: 포트가 사용 중인 경우 어떻게 하나요?

A: `Port` 설정을 다른 포트로 변경하거나 사용 중인 포트를 해제하세요.

Q: Newtonsoft.Json을 찾을 수 없습니다

A: `com.unity.nuget.newtonsoft-json` 패키지가 설치되어 있는지 확인:

```
Window > Package Manager > + > Add package by name
입력: com.unity.nuget.newtonsoft-json
```

Q: 도구에서 MonoBehaviour에 액세스하려면 어떻게 하나요?

A: `GameObject.Find()` 또는 싱글톤 패턴을 사용하여 씬의 객체 참조를 가져오세요.

Q: 도구 매개변수로 복잡한 객체를 전달하려면 어떻게 하나요?

A: 커스텀 타입을 정의하고 `[JsonProperty]`와 `[McpArgument]`로 필드를 어노테이션하세요.

五、AI 보조 개발

MCP For Unity는 AI 클라이언트를 통한 보조 개발을 지원합니다. 프로젝트 설정 파일을 작업 디렉토리에 복사하기만 하면 AI 어시스턴트가 프로젝트 구조를 이해하고 개발을 도울 수 있습니다.

5.1 빠른 시작

1단계: 설정 파일 복사

`AGENTS.md`를 프로젝트 루트에 복사:

```
# macOS / Linux
cp Assets/McpForUnity/AGENTS.md ./

# Windows
copy Assets\McpForUnity\AGENTS.md .
```

2단계: AI 클라이언트 시작

MCP 프로토콜을 지원하는 AI 클라이언트(OpenCode, Claude Desktop 등)로 프로젝트 디렉토리를 엽니다.

3단계: 개발 시작

AI 어시스턴트는 자동으로 `AGENTS.md`에서 프로젝트 정보를 읽습니다:

- 코드 규약 및 명명 규칙
- 프로젝트 구조 및 파일 위치
- 빌드 및 테스트 명령
- API 사용 방법

5.2 추천 도구

도구	설명
OpenCode	추천, 네이티브 MCP 프로토콜 지원
Claude Desktop	MCP 서버 설정 필요
Cursor	MCP 확장 지원
VS Code + MCP 플러그인	MCP 확장 설치 필요

5.3 개발 워크플로우

```
1. AI가 AGENTS.md 읽기 → 프로젝트 규약 이해
2. MCP Server 시작 → AI가 Unity 도구 호출 가능
3. 요구사항 설명 → AI가 코드 생성/수정
4. AI가 도구 호출 → 코드 검증/테스트
5. 반복 → 개발 완료
```

5.4 예제 시나리오

시나리오	예제 프롬프트
새 도구 추가	"씬 계층을 JSON으로 내보내는 에디터 도구 추가"
문제 수정	"EditorInstantiatePrefab 도구가 에러를 던집니다, 수정해 주세요"
코드 리팩토링	"이 도구 클래스를 프로젝트 규약에 따라 리팩토링"
테스트 작성	"EditorCreateGameObject에 대한 단위 테스트 작성"
문서 추가	"새 도구에 주석과 사용 문서 추가"

5.5 주의사항

- AI 어시스턴트는 MCP 프로토콜을 통해 Unity 에디터를 직접 조작할 수 있습니다
- AI가 코드를 수정한 후 컴파일 확인을 권장합니다
- 복잡한 작업은 단계별로 진행하여 검증 및 롤백을 쉽게 하세요

문서 버전: 1.0.3
최종 업데이트: 2026-03-02